Project 05 — Dual-Core RV32I SoC with Simplified Coherent Memory

0. What this document is

A frank, expert read-through of the project brief: what it actually asks for, what it deliberately does **not** ask for, where the hard parts hide, and how the 5 days realistically map to the deliverables. No fluff — the goal is that after reading this you understand the scope precisely enough to plan day-by-day without surprises.

---

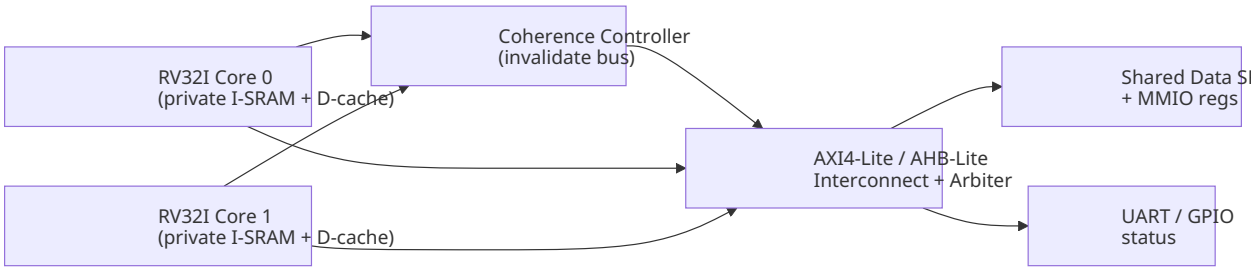
1. The one-sentence essence

> Build **two** simple RV32I cores that share one data memory through a **simplified invalidation-based coherence mechanism**, wrap it on a **bus (AXI4-Lite or AHB-Lite)**, simulate it (directed + UVM), synthesize it, run it through a **physical-design flow**, and boot it on an **FPGA** that shows coherence events on UART/LEDs.

Everything else in the brief is elaboration of that sentence. If you ever feel lost, come back to it.

---

2. The four pillars (and how they relate)



1. **Two RV32I cores** — the compute.
2. **Private caches + coherence controller** — the conceptual heart.
3. **Shared SRAM + bus arbiter** — the single source of truth.
4. **Verification + physical design + FPGA** — proof it works in silicon and in hardware.

The brief asks you to demonstrate **concepts**, not a production protocol. That distinction matters enormously (see §5).

---

3. Module-by-module — what is actually being graded

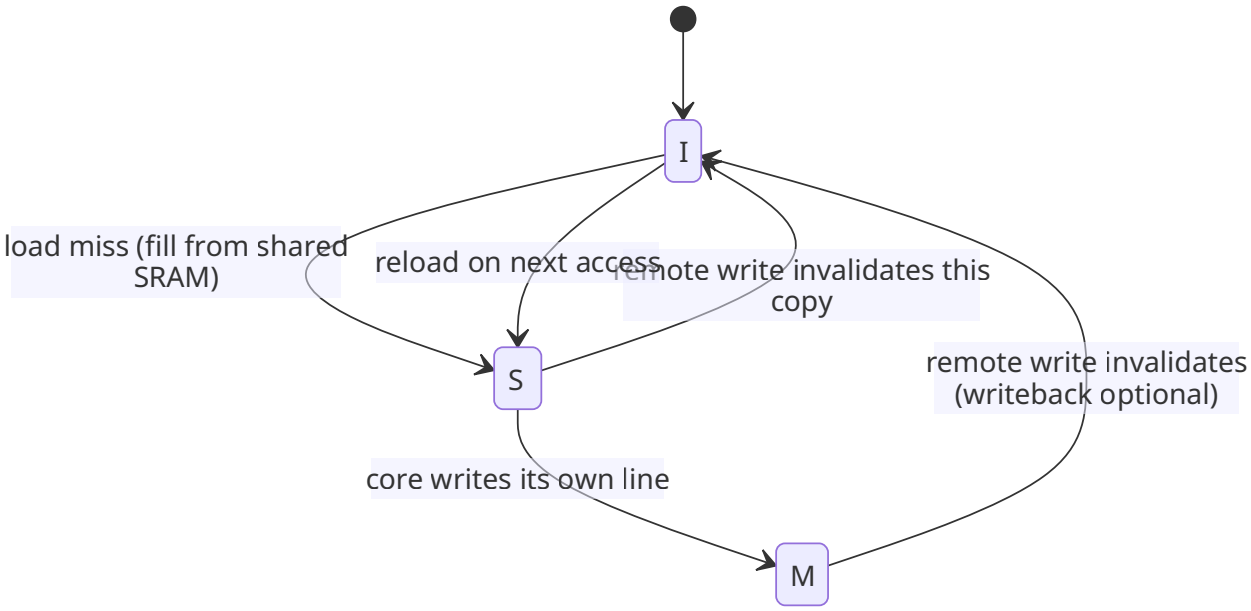
Module	What it really is	Hidden difficulty
Two RV32I cores	32-bit base integer ISA, single- or multi-cycle. You may reuse/integrate a known core.	Must be **synthesizable** and cycle-clean; bugs here poison everything downstream.

Module	What it really is	Hidden difficulty
Instruction SRAMs	Private per-core, preloaded with each core's program.	Trivial in sim, needs init (`$readmemh`/BRAM init) for FPGA.
Private data cache/ buffer	Small (a few lines). Tag + data + state per line.	The "state" field is where coherence lives.
Cache tag/data storage	Storage backing the above.	Keep it small — 2–4 lines is enough to demonstrate concepts.
Coherence controller	Drives invalidation on remote write; tracks line states.	**The intellectual core.** Not graded on protocol completeness, graded on *correctness of the demo*.
Invalidation logic	When core A writes a shared line, core B's copy is marked stale.	Must actually force a re-fetch, not just flip a bit nobody reads.
Shared data SRAM	One memory, one truth.	Single-port arbitration timing.
Arbiter / interconnect	Serialize two masters to one SRAM.	Round-robin is fine; fairness + deadlock-free matters.
AXI4-Lite / AHB-Lite	Bus fabric + control interface.	AXI-Lite's 5 channels add verbosity; AHB-Lite is simpler but less realistic. Pick deliberately.
Coherence status/ debug regs	MMIO-visible state for SW/debug.	Cheap to add, big leverage on the FPGA demo.
UART / GPIO	Human-visible proof.	UART is the single most reliable "it works" signal on FPGA.

---

4. The coherence model they actually want

They explicitly say: **"A complete industry-level coherence protocol is not required."** So do **not** burn time implementing MESI/MOESI with snooping buses, directories, M-state writebacks, etc. Build the **minimal mechanism that demonstrates the concept**:



Three states are enough: **I** (Invalid / not present), **S** (Shared/clean copy), **M** (Modified/dirty, this core owns it). On a write by core A:

- Core A moves its line to **M** and writes shared SRAM (write-through is simplest and defensible).

- Coherence controller signals core B: "invalidate line X."
- Core B marks its copy **I** → next read refetches the updated value.

That single flow is the entire "demonstrate coherence" requirement. Everything else is engineering polish.

Write-through vs write-back: For a 5-day student project, **write-through + invalidate-on-write** is the pragmatic choice. Write-back adds dirty-tracking and writeback-on-eviction complexity that buys you nothing the grader asked for.

---

5. What the brief *deliberately excludes* (read this twice)

> "Pipelining, advanced cache hierarchies, and scalable multi-core coherence protocols are **not required**."

This is permission to stay small. Translated:

- **No pipeline** — single-cycle or multicycle core is fine. Don't build a 5-stage pipeline; it will eat your schedule and gain no marks.
- **No L2 / multi-level cache.** One tiny level of private cache per core.
- **No scalable snooping/directory bus.** Two cores, hardwired invalidate path.
- **No interrupts, no exceptions, no privileged CSRs** beyond what you need for debug regs. RV32I base = no CSR instr anyway.
- **No DRAM controller** — SRAM only.

If you spend effort on any excluded item, that effort is **wasted**. The brief rewards concept-demonstration, not feature-count.

---

6. The three verification layers (don't conflate them)

Layer	Tool	Purpose	What "done" looks like
Basic	Directed self-checking SV TB	Prove the specific coherence story works end-to-end.	A test that writes from core 0, reads from core 1, asserts the new value appears *after* invalidation. Plus reset + conflict cases.
UVM	Agents, seqs, scoreboard, coverage	Randomized stress of concurrent accesses.	Scoreboard predicts expected shared-memory state; coverage hits hit/miss, invalidation, simultaneous-write, error cases.
FPGA	Hardware	Visible proof on LEDs/UART.	LED blinks per core/coherence event; UART prints the before/after shared value.

These are **three separate deliverables**, not one big test. The directed TB is your fallback truth — if UVM gets shaky on day 4, the directed TB still proves the core behavior.

---

7. Physical design — the part most students underestimate

"Run synthesis and the assigned physical-design flow" is **not** optional polish; it's a graded deliverable with concrete outputs:

- SDC constraints (clock definition, I/O delays)

- Floorplan → power plan → placement → CTS → routing → sign-off
- **Reports you must produce:** area, timing (setup/hold WNS/TNS), power, congestion, **DRC clean**, **LVS clean**, final layout screenshots

The trap: physical design only works on **clean, synthesizable, constrained RTL**. A core that simulates but fails synthesis (latches, async resets inferred wrong, uninitialized RAMs, combinational loops) will block the back-end flow for a whole day. **Fix RTL quality early.**

---

8. FPGA demo — what "demonstrate" concretely means

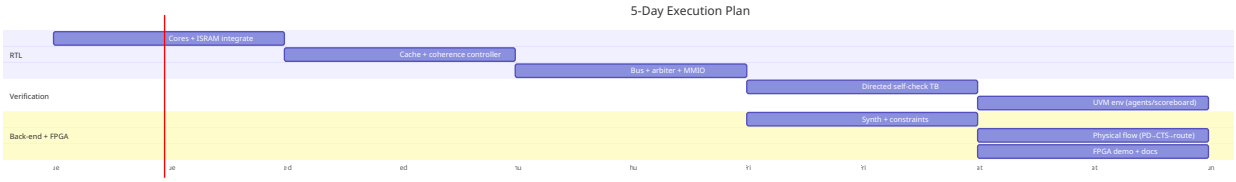
The grader wants to *see* coherence happen. Plan for:

- Core 0 writes a known value to shared addr X (LED/UART shows "C0 write X").
- Coherence event fires (LED pulse / UART line "INVALIDATE → C1").
- Core 1 reads X and gets the **new** value (LED/UART shows it).
- A **mismatched/stale** read would fail this — so the demo doubles as a live self-check.

UART is more convincing than LEDs alone because it logs the sequence. Use both: LEDs for "alive / event" heartbeat, UART for the actual values.

---

9. Realistic 5-day plan (with honest risk callouts)



Risk-weighted reality check:

- **Day 1 (cores + ISRAM)** — highest leverage. A working RV32I core is the foundation. *Risk:* debugging the CPU eats the day. *Mitigation:* use a known-good single-cycle core design; don't reinvent.
- **Day 2 (cache + coherence)** — the conceptual heart, but mechanism is small. *Risk:* over-engineering the protocol. *Mitigation:* 3-state I/S/M, write-through, hardwired invalidate.
- **Day 3 (bus + MMIO)** — glue + debug regs. *Risk:* AXI-Lite handshake bugs. *Mitigation:* AHB-Lite if time-pressured; it's a valid choice per brief.
- **Day 4 (directed TB + UVM)** — verification. *Risk:* UVM env scope creep. *Mitigation:* directed TB must be done first and solid; UVM is layered on top.
- **Day 5 (synth + PD + FPGA)** — back-end + demo. *Risk:* synthesis/physical flow failures eat the day. *Mitigation:* run a **synthesis smoke test on day 2-3** so day 5 isn't the first time tools see your RTL.

The single biggest schedule killer is **leaving synthesis until the last day.** Do a clean synthesis check as soon as the top RTL exists, even if timing isn't met yet.

---

10. Suggested top-level signal list (to anchor the design)

- `clk`, `rst_n` (async, active-low — pick one reset style and use it everywhere)

- `core0_irq` / `core1_irq` — optional, likely unused
- MMIO region for: coherence status reg, event counters (invalidations, hits, misses), core-to-core doorbell
- UART TX/RX pins, LED outputs
- (FPGA) clock constraints from the board's reference clock, possibly with a PLL/MMCM for core clk

---

11. Decisions you must make on day 1 (they ripple all week)

1. **Core style:** single-cycle vs multicycle. (Recommendation: single-cycle for predictability, if your PDK timing can close.)
2. **Bus:** AXI4-Lite vs AHB-Lite. (Recommendation: AHB-Lite if you're time-pressed; AXI-Lite if the course expects it.)
3. **Write policy:** write-through vs write-back. (Recommendation: write-through.)
4. **Coherence states:** I/S/M (3) vs full MESI. (Recommendation: I/S/M.)
5. **Cache size:** 2–4 lines, direct-mapped. (Don't go bigger.)
6. **Reset domain:** single async reset, synchronized — one policy, used consistently.
7. **Toolchain:** which synthesizer/PDK (course-assigned), which FPGA board.

These seven decisions, written down, prevent mid-week rework.

---

12. The "transparent" warnings (what the brief doesn't say but you must know)

- **Coherence correctness is subtle even when "simplified."** The classic bug: core B's cache marks a line invalid but core B **already has the value in a register** from a prior load — your coherence only fixes *future* loads. The demo program must be written so the value is re-read *after* the invalidation, not cached in an architectural register. Design the demo SW around this.
- **Atomicity of "simultaneous writes."** Real simultaneous writes to the same line need an arbitration winner + loser invalidation. The brief lists "simultaneous writes" as a test case — your arbiter must define a deterministic winner.
- **"Shared memory" vs "shared data."** Code (I-SRAM) is private per core; only **data** is shared. Don't accidentally make instruction fetches go through the coherence path.
- **Reset behavior.** All caches → Invalid, all state regs → known values. A directed reset test is explicitly required.
- **Error conditions.** The brief lists "error conditions" as a test case. Decide what your errors *are* (e.g., bus error response, invalid access to unmapped region) and handle them — don't leave them undefined, or UVM coverage will expose the gap.
- **Coverage is a deliverable.** Code coverage + functional coverage are both named. Don't skip adding coverpoints for the coherence states and events.

---

13. Minimum viable demo program (conceptual)

```
# Core 0 # Core 1
sw r1, 0(shared) # writes X (spin) lw r2, 0(shared) # expects X after invalidation
 # coherence controller invalidates C1's line first
```

```
  # so this lw refetches = sees X, not stale
```

If this exact scenario passes in directed sim **and** on the FPGA, the project's core thesis is demonstrated. Everything else is breadth.

---

14. Bottom line

- **Scope:** two small RV32I cores + tiny private caches + a 3-state invalidate-on-write coherence link + one shared SRAM on a Lite bus, verified three ways, pushed through synthesis/physical-design, and shown on FPGA.
- **Intellectual center of gravity:** the coherence controller and the demo that proves stale data gets invalidated.
- **Schedule risk centers:** day 1 (CPU), day 5 (back-end). Both are mitigated by reusing a clean core and running an early synthesis smoke test.
- **The brief gives you permission to stay small.** Use it. A small, correct, fully-verified, cleanly-physical-design'd design beats a big half-finished one every time.

Next step: confirm the seven Day-1 decisions (§11) and the assigned toolchain/PDK/FPGA, and I can turn this analysis into a concrete module-spec + day-by-day task list.